

CC3092 – Deep Learning y Sistemas Inteligentes · Hoja de trabajo #2: Transformers y mecanismos de atención

2. De las RNN a la atención

2.1 Cuello de botella y atención de Bahdanau. En el seq2seq con RNN el encoder comprime toda la oración en un único vector de tamaño fijo (el último estado oculto), que es lo único que recibe el decoder. Su capacidad no crece con la longitud y la información del inicio debe sobrevivir muchos pasos de recurrencia (olvido, vanishing gradient, etc.), por lo que la calidad cae con oraciones largas (Cho et al., 2014). Bahdanau et al. (2014) conservan todos los estados h_j del encoder bidireccional y en cada paso i calculan $e_{ij} = v^T \tanh(W s_{i-1} + U h_j)$, $\alpha_{ij} = \text{softmax}_j(e_{ij})$ y un contexto propio $c_i = \sum_j \alpha_{ij} h_j$. El decoder «consulta» directamente las posiciones relevantes, el contexto deja de ser fijo, se crea un atajo para el gradiente y se obtiene un alineamiento interpretable.

2.2 Aditiva vs. multiplicativa. La aditiva (Bahdanau) puntúa con una pequeña red, $\text{score}(q,k) = v^T \tanh(W_1 q + W_2 k)$, con parámetros W_1, W_2, v ; la multiplicativa (Luong) usa un producto interno $q^T k$ (*dot*) o $q^T W k$ (*general*). La multiplicativa es más eficiente en hardware reciente porque todos los scores salen de una sola multiplicación de matrices QK^T (GEMM) (muy optimizada en GPU) (tensor cores, cuBLAS), mientras que la aditiva debe materializar un tensor $n \times m \times d$ y aplicar tanh elemento a elemento. La complejidad teórica es similar, pero el producto punto sin escalar rinde peor con d_k grande, lo que motiva el factor $1/\sqrt{d_k}$ (Vaswani et al., 2017).

2.3 Analogía Q–K–V. Un diccionario devuelve el valor cuya llave coincide *exactamente* con la consulta; la atención es una búsqueda *suave* y diferenciable: la consulta se compara con todas las llaves, la softmax convierte las similitudes en pesos que suman 1 y se devuelve el promedio ponderado de los valores (Q = «qué busco», K = «cómo me encuentran», V = «qué entrego»). En self-attention Q, K y V salen de la misma secuencia X por proyecciones aprendidas ($Q = XW^Q, K = XW^K, V = XW^V$); en cross-attention Q viene del decoder y K, V de la salida del encoder.

2.4 Limitaciones de las RNN que se eliminan. (i) *Procesamiento secuencial*: h_t depende de h_{t-1} ($O(n)$ operaciones secuenciales); la self-attention calcula todas las posiciones a la vez ($O(1)$). (ii) *Dependencias largas*: entre dos tokens a distancia d la RNN necesita $O(d)$ pasos; la atención los conecta en un salto (camino $O(1)$). (iii) *Desvanecimiento del gradiente*: la retropropagación en el tiempo multiplica muchos jacobianos $\partial h_t / \partial h_{t-1}$; con atención, residuales y LayerNorm el gradiente fluye por caminos cortos. El precio es un costo $O(n^2 \cdot d)$ por capa frente a $O(n \cdot d^2)$ de la RNN (Vaswani et al., 2017, Tabla 1).

3. La arquitectura Transformer

Componentes ($N = 6, d_{\text{model}} = 512, h = 8, d_{\text{ff}} = 2048$): embeddings + codificación posicional (vectores de dimensión d_{model} con información de orden); encoder (N capas de self-attention multi-cabeza + FFN, con residual y LayerNorm) que produce la representación contextual de la fuente; decoder (self-attention causal + cross-attention sobre el encoder + FFN) y capa final lineal + softmax sobre el vocabulario.

3.1 Scaled dot-product attention: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$. Si q_i, k_i son independientes con media 0 y varianza 1, cada producto $q_i k_i$ tiene media 0 y varianza 1, y $\text{Var}(q \cdot k) = \sum_i \text{Var}(q_i k_i) = d_k$ (desviación $\sqrt{d_k}$; ≈ 8 con $d_k = 64$). Sin escalar, los logits crecen con d_k , la softmax se satura (casi *one-hot*) y su jacobiano $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j) \rightarrow 0$: los gradientes se desvanecen. Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1 para cualquier d_k (verificado en §5.1 y Fig. 1a–b).

3.2 Multi-head attention. h cabezas de dimensión d_{model}/h aprenden proyecciones distintas, así que cada una atiende a un subespacio y a un tipo de relación diferente (posicional, sintáctica, correferencia) a la vez; con una sola cabeza, un único promedio ponderado mezcla esas relaciones. Las salidas se concatenan y se proyectan con W^O . El número de parámetros no cambia: las proyecciones suman d_{model}^2 por cada uno de Q, K, V más d_{model}^2 de W^O , es decir $4 \cdot d_{\text{model}}^2$ (+ sesgos) para cualquier h (verificado en §5.1); lo que crece es la memoria de las h matrices $n \times n$.

3.3 Codificación posicional. La self-attention es equivariante a permutaciones (permutar la entrada solo permuta la salida, §5.1), por lo que sin información extra no distingue «el perro muerde al hombre» de «el hombre muerde al perro». **Sinusoidal:** $PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d})$, $PE_{(\text{pos}, 2i+1)} = \cos(\text{pos}/10000^{2i/d})$; longitudes de onda de 2π a $10000 \cdot 2\pi$, sin parámetros, y $PE_{\text{pos}+k}$ es una rotación lineal de PE_{pos} , lo que facilita atender por posición relativa. **Aprendida** (BERT, GPT-2): tabla $L_{\text{max}} \times d$ entrenada con el modelo; rinde igual, pero no existe más allá de L_{max} (512 en BERT, 1024 en GPT-2). **RoPE** (Su et al., 2021): rota q y k por pares de dimensiones con un ángulo proporcional a la posición, de modo que $q_m \cdot k_n$ depende solo de $m - n$; no añade parámetros y se extiende a contextos largos con escalado (PI, YaRN); la usan LLaMA, Mistral y Qwen.

3.4 Bloque completo. La conexión residual $x + \text{Sublayer}(x)$ da un camino de identidad al gradiente; LayerNorm normaliza cada token sobre sus características (independiente del batch); la FFN por posición $\max(0, xW_1 + b_1)W_2 + b_2$ aplica los mismos pesos a cada posición: la atención mezcla información *entre* posiciones y la FFN la transforma *dentro* de cada una.

	Post-LN (artículo original)	Pre-LN (modelos actuales)
Fórmula	$x \leftarrow \text{LN}(x + \text{Sublayer}(x))$	$x \leftarrow x + \text{Sublayer}(\text{LN}(x)), + \text{LN final}$
Camino residual	La LN está sobre el camino principal: se re-normaliza en cada capa.	Identidad limpia de la entrada a la salida.
Entrenamiento	Gradientes grandes cerca de la salida al inicio; requiere warm-up y es frágil en redes profundas (Xiong et al., 2020).	Estable sin warm-up o con uno corto; estándar en GPT-2/3 y LLaMA (con RMSNorm), aunque puede subutilizar capas profundas.
Notebook (B.3)	Gradientes $\approx 5\text{--}8\times$ menores a la inicialización (12 capas).	Gradientes mayores en todas las capas.

3.5 Complejidad. Calcular QK^T cuesta $O(n^2 \cdot d)$ en tiempo y la matriz de atención ocupa $O(n^2)$ de memoria (por cabeza, capa y ejemplo); el resto de las operaciones es lineal en n , así que el término cuadrático domina en secuencias largas (medido en Fig. 1c–d: pendientes 2.10 y 1.90).

Propuesta	Idea	Qué sacrifica
FlashAttention (Dao et al., 2022)	Cálculo exacto por bloques (tiling) con softmax en línea en SRAM, sin escribir la matriz $n \times n$ en HBM: memoria $O(n)$.	No reduce los FLOPs (siguen siendo $O(n^2)$); requiere kernels específicos de hardware.
Ventanas / dispersa (Longformer, BigBird)	Cada token atiende a w vecinos (+ tokens globales o aleatorios): costo $O(n \cdot w)$.	Pierde la conexión directa entre tokens lejanos (solo se recupera apilando capas o con tokens globales).
Atención lineal (Katharopoulos et al., 2020; Performer)	Sustituye la softmax por un mapa ϕ y reasocia $\phi(Q)(\phi(K)^T V)$: costo $O(n \cdot d^2)$.	Aproxima la softmax; peor recuperación exacta o «afilada» de información.

4. Atención en PyTorch y Hugging Face

Función / clase	Propósito y parámetros relevantes
F.scaled_dot_product_attention	Calcula $\text{softmax}(QK^T \cdot \text{scale})V$ con kernels fusionados (Flash, memory-efficient o matemático según el hardware). <code>attn_mask</code> : booleana (True = participa) o flotante que se suma a los scores (−inf bloquea). <code>dropout_p</code> : dropout sobre los pesos (pasar 0.0 en evaluación). <code>is_causal</code> : máscara triangular inferior. <code>scale</code> : por defecto $1/\sqrt{E}$.
nn.MultiheadAttention	Capa multi-cabeza con proyecciones Q/K/V/O. <code>embed_dim</code> (divisible entre <code>num_heads</code> ; cada cabeza usa <code>embed_dim/num_heads</code>), <code>batch_first</code> (entradas (N, L, E) en vez de (L, N, E)), <code>kdim/vdim</code> (dimensión de K y V si difieren, p. ej. en cross-attention). En forward: <code>key_padding_mask</code> (N, S), True = ignorar esa llave; <code>attn_mask</code> (L, S) o (N·h, L, S), aquí True = bloqueado (convención opuesta a SDPA); <code>need_weights=False</code> evita devolver pesos (ruta rápida); <code>average_attn_weights</code> promedia sobre cabezas (N, L, S) o no (N, h, L, S).
nn.TransformerEncoderLayer	Bloque encoder completo (self-attention + FFN + residuales + LayerNorm). <code>d_model</code> , <code>nhead</code> , <code>dim_feedforward</code> (ancho de la FFN, 2048 por defecto), <code>dropout</code> (0.1), <code>norm_first</code> (False = Post-LN, True = Pre-LN).
nn.Transformer.generate_square_subsequent_mask (sz)	Método estático que devuelve una matriz $sz \times sz$ flotante con −inf sobre la diagonal y 0 en el resto: máscara causal para el decoder.
Hugging Face: output_attentions=True	El modelo devuelve <code>outputs.attentions</code> : una tupla con un tensor por capa de forma (batch, cabezas, n, n); la fila es la consulta y suma 1. Requiere <code>attn_implementation="eager"</code> (sdpa/Flash no devuelven pesos). BERT-base y GPT-2 small: 12 capas × 12 cabezas.

5. Resultados experimentales

Todos los resultados provienen del notebook `Hoja2_Transformers_Atencion.ipynb` (GPU, bert-base-uncased y gpt2 con `attn_implementation="eager"`).

5.1 Verificaciones numéricas, escalamiento y costo computacional

Verificación	Resultado
Var($q \cdot k$), $q, k \sim N(0, I)$	$d_k = 4 / 16 / 64 / 256 / 1024 \rightarrow 3.98 / 16.07 / 64.25 / 253.07 / 1015.56$ (teórico d_k); tras $\pm \sqrt{d_k}$: 0.99–1.00.
Saturación de la softmax (16 llaves)	Sin escalar, $p_{\max}$ sube de 0.43 a 0.96 y $\ J\ $ cae de 0.32 a 0.05 ($6\times$) al pasar d_k de 4 a 1024. Escalada: $p_{\max} \approx 0.25$ y $\ J\ \approx 0.30$ constantes.
Parámetros de multi-head ($d_{\text{model}} = 512$)	$1\,050\,624 = 4 \cdot 512^2 + 4 \cdot 512$ para $h = 1, 2, 4, 8$ y 16: no dependen de h .
Equivarianza a permutaciones	$\max SA(PX) - P \cdot SA(X) = 8.9 \times 10^{-8}$ (sin codificación posicional).
PE sinusoidal	$PE_p \cdot PE_{p+5} = 47.185$ para $p = 0, 10$ y 50: solo depende de la distancia relativa.
SDPA y nn.MultiheadAttention	SDPA = implementación manual con y sin <code>is_causal</code> ; máscara booleana <code>tril</code> $\equiv$ <code>is_causal</code> ; <code>scale=1.0</code> $\equiv$ sin escalar. MHA: atención a llaves de padding = 0 y filas que suman 1.
Costo (GPU, 8 cabezas, $d = 64$)	Pendiente log-log para $n \geq 1024$: tiempo 2.10 , memoria pico 1.90 (teórico 2). En $n = 8192$ la matriz ocupa 2 GiB (pico 4.1 GiB); SDPA $\approx 2\times$ más rápida pero también cuadrática (pendiente 1.97).

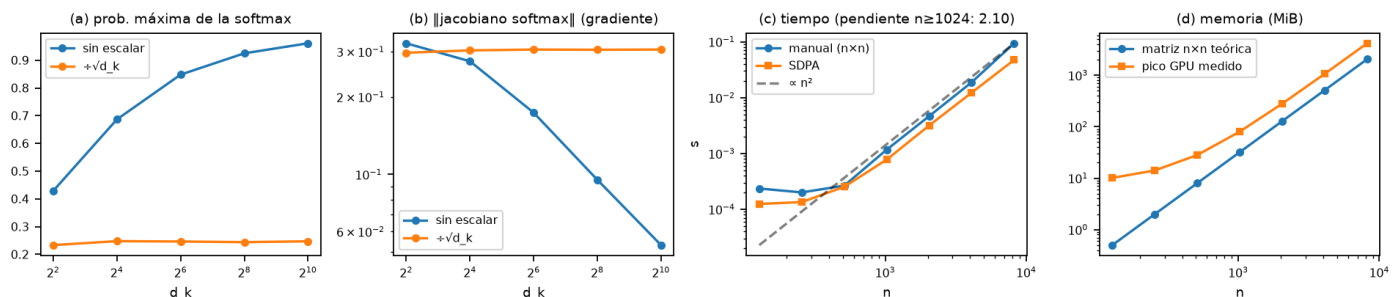


Figura 1. (a–b) Efecto del factor $1/\sqrt{d_k}$ sobre la softmax y su gradiente. (c–d) Tiempo y memoria de la self-attention vs. n (log-log); para $n < 1024$ domina el costo fijo de lanzar kernels en la GPU.

5.2 Mapas de atención en BERT y GPT-2

Oración: «The cat sat on the mat, and then it jumped onto the table.» Para elegir las cabezas se midió, en las 144 de cada modelo, la atención media al token anterior/siguiente, a sí mismo, al primer token/[CLS], a [SEP] y a la puntuación (Fig. 2). **BERT**: la capa 3-cabeza 10 atiende al token *siguiente* (0.94) y la capa 4-cabeza 6 al *anterior* (0.68); la capa 6-cabeza 2 envía 0.84 de la masa a [SEP] y la capa 12-cabeza 8 0.89 a «.» y «.». Con umbral 0.5, 43 cabezas apuntan a [SEP], 18 a la puntuación, 5 al siguiente, 4 a [CLS], 3 al anterior y ninguna a sí misma. **GPT-2**: la capa 1-cabeza 2 atiende a *sí misma* (0.94), la capa 5-cabeza 12 al *anterior* (1.00) y la capa 6-cabeza 2 envía todo al primer token; 99 de 144 cabezas superan 0.5 en el primer token y ninguna atiende a la puntuación. Todo lo que está sobre la diagonal es 0 (máscara causal). Las columnas verticales en [SEP], puntuación o primer token funcionan como «no-op»: la cabeza no encuentra nada útil y deposita la masa ahí (Clark et al., 2019).

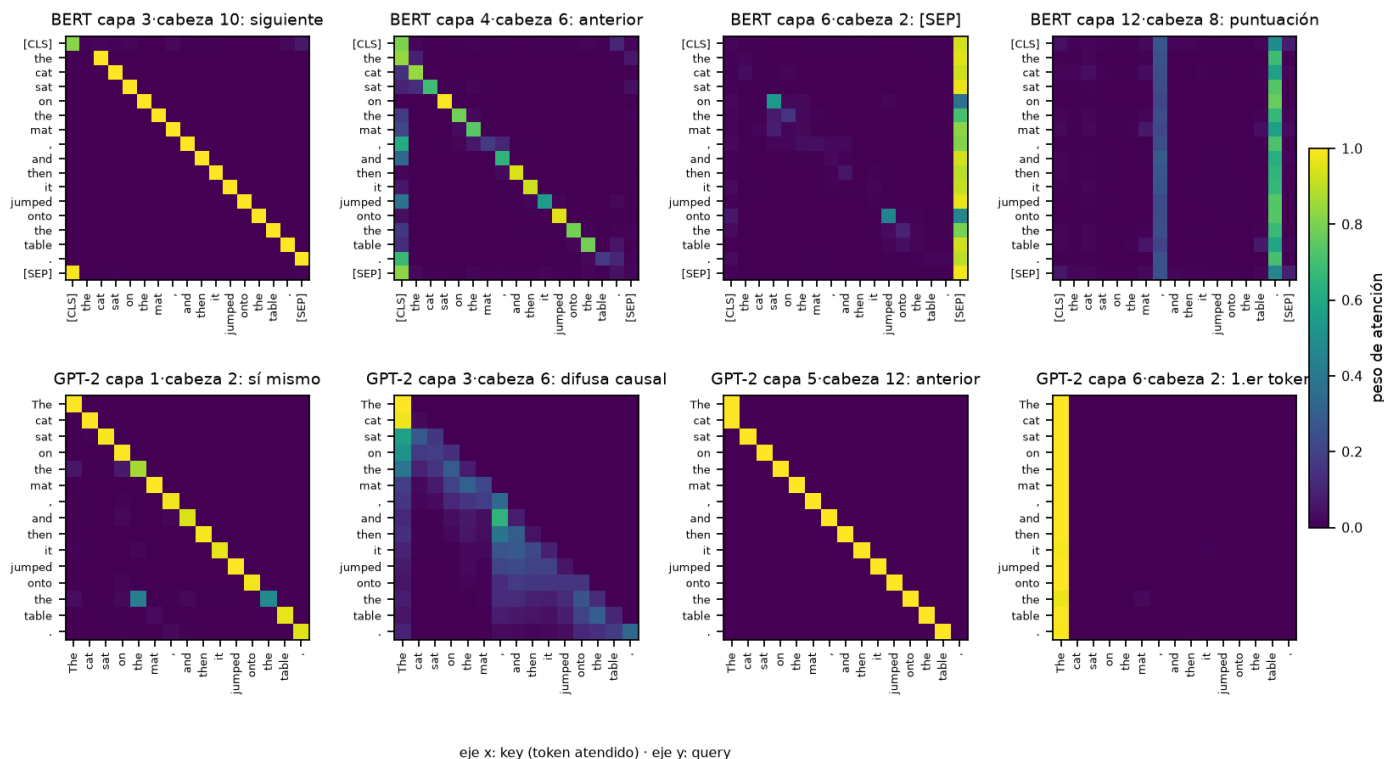


Figura 2. Pesos de atención de 4 cabezas de capas distintas en BERT (arriba) y GPT-2 (abajo). Filas = query, columnas = key.

5.3 «it» → animal o street (BERT)

Para cada cabeza se calculó $s = a(it \rightarrow \text{animal}) - a(it \rightarrow \text{street})$ en «...because it was too *tired*» (A) y «...too *wide*» (B), y $\Delta = s_A - s_B$. La **capa 7-cabeza 11** es la más sensible al contexto ($\Delta = 0.80$): con *tired* «it» atiende más a *animal* (0.28 vs. 0.07) y con *wide* a *street* (0.59 vs. 0.01) (Fig. 3a–b). Como con 144 cabezas y un solo par un Δ grande podría ser azar, se probaron 6 pares tipo Winograd (trophy/suitcase, horse/fence, car/hill, bird/seed, dog/river y el original): esa cabeza elige el sustantivo correcto en **10/12** oraciones (azar = 6/12), mientras que otras con Δ alto en el par original no generalizan (capa 9-cabeza 11: 7/12). Solo 8 de 144 cabezas llegan a $\geq 10/12$.

5.4 GPT-2: máscara causal y attention sinks

En un texto de 45 tokens el triángulo superior vale exactamente 0 en las 144 cabezas: **la matriz es triangular inferior**. La proporción media de atención que recibe el primer token (consultas $i \geq 1$) por capa es 0.09, 0.19, 0.23, 0.44, 0.48, 0.66, 0.65, **0.74**, 0.65, 0.70, 0.66 y 0.50 (capas 1–12), frente a 0.08 si fuera uniforme (Fig. 3c). Al cambiar la primera palabra por «Banana» o «.» los valores varían ≤ 0.02 : depende de la *posición*, no del contenido. Es el fenómeno de *attention sinks* (Xiao et al., 2023): como cada fila de la softmax debe sumar 1, el modelo deposita la atención «sobrante» en el primer token, visible para todas las consultas; por eso StreamingLLM conserva esos tokens en la caché KV junto a una ventana deslizante. En BERT, [CLS] + [SEP] cumplen ese papel en las capas 6–9 (≈ 0.55), pero no en las 11–12 (≤ 0.06).

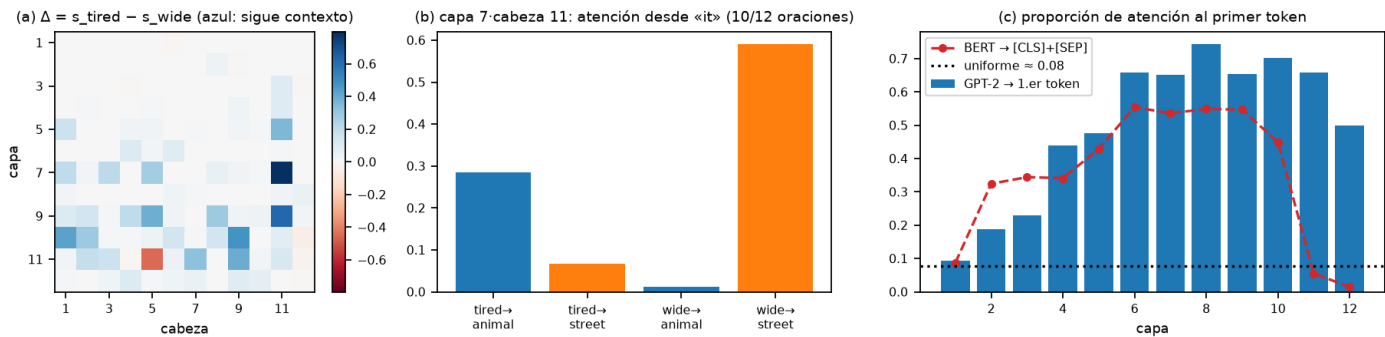


Figura 3. (a) Δ por cabeza en BERT; (b) atención desde «it» en la mejor cabeza; (c) atención al primer token por capa en GPT-2 vs. [CLS]+[SEP] en BERT.

5.5 Entropía de la atención por capa

Se usó $H = -\sum_j a_{ij} \log a_{ij}$, normalizada por $\log(\#\text{llaves visibles})$ para comparar el modelo causal con el bidireccional (1 = uniforme), promediada sobre 8 oraciones (Fig. 4). **BERT** baja de 0.77 (capa 1) a 0.40 (capa 8) y vuelve a subir a 0.56 en la capa 12; **GPT-2** baja de 0.74 (capa 2) a 0.30 (capas 10–11) y sube a 0.45 en la capa 12. Las capas profundas atienden de forma **más concentrada** que las iniciales, con un repunte al final. GPT-2 es más concentrado que BERT desde la capa 4, pero en buena parte por el *sink*, y dentro de una misma capa conviven cabezas casi uniformes y casi deterministas.

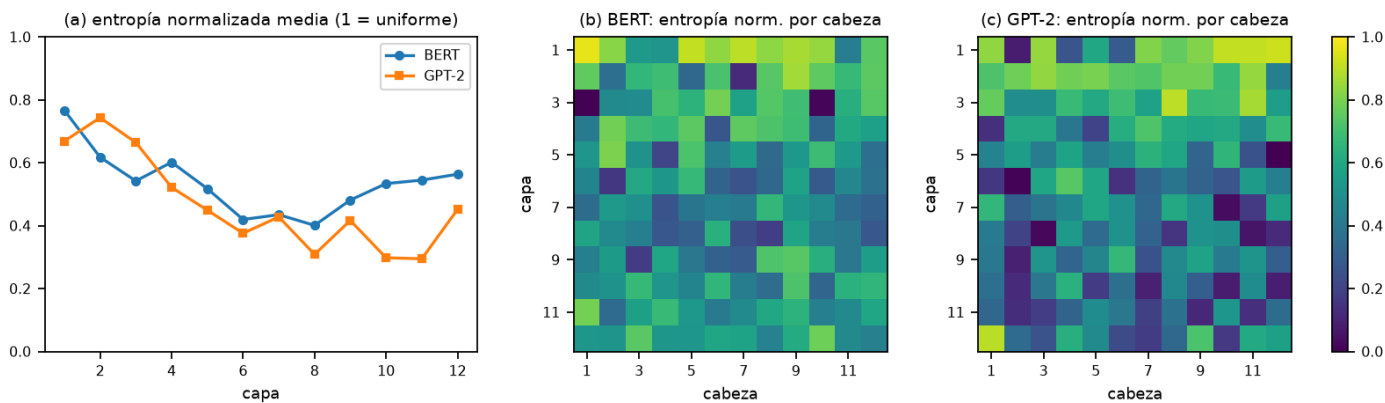


Figura 4. (a) Entropía normalizada media por capa; (b–c) entropía normalizada por cabeza en BERT y GPT-2.

6. Discusión y análisis

¿Por qué la atención permitió modelos mucho más grandes? (i) *Paralelismo*: sin recurrencia todas las posiciones se procesan a la vez y el entrenamiento escala con el hardware; la RNN necesita n pasos secuenciales. (ii) *Camino corto*: cualquier par de tokens está a un salto, lo que facilita las dependencias largas y estabiliza el gradiente. (iii) *Costo cuadrático*: es el precio (pendiente ≈ 2 medida), pero se paga con multiplicaciones de matrices densas muy eficientes y se mitiga con kernels como FlashAttention (SDPA fue $\approx 2\times$ más rápida); en la práctica el cuello de botella pasó de la latencia secuencial a la memoria.

¿Las cabezas se especializan? ¿Todas son útiles? Sí hay especialización clara y estable: cabezas posicionales casi perfectas (siguiente 0.94, anterior 1.00), cabezas de delimitadores y puntuación, y una cabeza de correferencia (capa 7·cabeza 11, 10/12). Pero muchas parecen «no-op»: 43 cabezas de BERT concentran $> 50\%$ en [SEP] y 99 de GPT-2 en el primer token. Esto coincide con Voita et al. (2019) y Michel et al. (2019), que podan la mayoría de las cabezas con poca pérdida: no todas son igual de útiles.

¿Los pesos de atención explican las decisiones? Solo parcialmente. Jain y Wallace (2019) muestran que los pesos correlacionan poco con la importancia por gradientes y que existen distribuciones muy distintas con la misma predicción; Wiegrefe y Pinter (2019) matizan que esas distribuciones adversarias no salen de entrenar el modelo y que la atención puede dar explicaciones *plausibles*, aunque no necesariamente *fieles*. Nuestros resultados apoyan esa postura intermedia: el patrón más masivo (sink/[SEP]) no tiene contenido semántico (cambiar la primera palabra no lo altera), la baja entropía de capas profundas se debe en parte a él, y la cabeza 7·11 solo *correlaciona* con la resolución de «it». Además, los pesos ignoran la norma de los vectores de valor (Kobayashi et al., 2020) y se mezclan a través de capas y residuales; para afirmar causalidad harían falta intervenciones (ablación de cabezas, gradientes).

¿Cuándo preferir una RNN o una CNN? (a) Inferencia en *streaming* o en dispositivos con poca memoria: la RNN tiene un estado de tamaño constante y costo $O(1)$ por token, sin caché KV creciente. (b) Pocos datos o poco cómputo: los sesgos inductivos (localidad de la CNN, recurrencia de la RNN) mejoran la eficiencia muestral. (c) Señales con estructura local (audio, series temporales, visión en el borde), donde una CNN es más barata. Modelos como Mamba o RWKV retoman la idea recurrente por estas ventajas.

Documentos de 100 000 tokens. Con atención estándar la matriz tiene $n^2 = 10^{10}$ entradas por cabeza y capa: ≈ 20 GB en fp16 por cabeza si se materializa (extrapolando nuestra medición de 2 GiB en $n = 8192$, serían ≈ 300 GiB para 8 cabezas en fp32), además de $\sim 10^{10}$ multiplicaciones-suma por cabeza y capa y una caché KV que crece con n . Los embeddings

posicionales aprendidos no llegan a esa longitud y la calidad cae en contextos largos («lost in the middle»). Alternativas: FlashAttention (exacta, memoria $O(n)$), atención por ventanas/dispersa (Longformer, BigBird), atención lineal o SSM (Mamba), RoPE con escalado (PI, YaRN), GQA/MQA para reducir la caché KV, *ring attention* entre GPUs, o estrategias de sistema como dividir en fragmentos, resumen jerárquico o RAG.

7. Conclusiones

- El escalamiento $1/\sqrt{d_k}$ es necesario: se verificó que $\text{Var}(q \cdot k) = d_k$ y que sin él la softmax se satura ($p_{\max} = 0.96$ con $d_k = 1024$) y su gradiente cae $6\times$; con él, ambas magnitudes son independientes de d_k .
- El costo de la self-attention es cuadrático en la práctica (pendientes 2.10 en tiempo y 1.90 en memoria). Los kernels fusionados (SDPA) reducen la constante $\approx 2\times$, pero no el orden; para contextos largos se necesitan atención dispersa/lineal o SSM.
- Las cabezas se especializan (posicionales, delimitadores, puntuación y una de correferencia que generaliza a 10/12 oraciones), pero muchas actúan como «no-op»: la atención se concentra en [SEP] o en el primer token.
- GPT-2 muestra *attention sinks* claros (hasta 74 % de la atención en el primer token, independiente de su contenido) y la entropía baja con la profundidad en ambos modelos; por eso los pesos de atención son una herramienta de inspección útil, pero no una explicación causal.

Referencias

Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
Bahdanau, D., Cho, K., Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473.
Luong, M.-T., Pham, H., Manning, C. (2015). Effective Approaches to Attention-based Neural Machine Translation. arXiv:1508.04025.
Cho, K. et al. (2014). On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. arXiv:1409.1259.
Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture. arXiv:2002.04745.
Dao, T. et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135.
Beltagy, I., Peters, M., Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.
Katharopoulos, A. et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. arXiv:2006.16236.
Xiao, G. et al. (2023). Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.
Clark, K. et al. (2019). What Does BERT Look At? An Analysis of BERT's Attention. arXiv:1906.04341.
Voita, E. et al. (2019). Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. arXiv:1905.09418.
Michel, P., Levy, O., Neubig, G. (2019). Are Sixteen Heads Really Better than One? arXiv:1905.10650.
Jain, S., Wallace, B. (2019). Attention is not Explanation. arXiv:1902.10186.
Wiegrefe, S., Pinter, Y. (2019). Attention is not not Explanation. arXiv:1908.04626.
Kobayashi, G. et al. (2020). Attention is Not Only a Weight: Analyzing Transformers with Vector Norms. arXiv:2004.10102.
Gu, A., Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
Documentación de PyTorch (scaled_dot_product_attention, nn.MultiheadAttention, nn.TransformerEncoderLayer) y de Hugging Face Transformers.

Repositorio (Git): https://github.com/Ren23813/HDT2_Deep